

**МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ**

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ

**по дисциплине «Введение в нереляционные базы
данных»**

**Тема: Симуляция уборки снега — графовая модель
дорожной сети, маршрутизация снегоуборочной техники,
учёт поломок и расхода топлива.**

Студенты гр. 3343

Старков С. А.

Поддубный В.А

Жучков О. Д.

Преподаватель

Заславский М.М.

Санкт-Петербург

ЗАДАНИЕ

Студенты

Старков С. А

Поддубный В.А.

Жучков О. Д.

Группа 3343

Тема проекта: Симуляция уборки снега — графовая модель дорожной сети, маршрутизация снегоуборочной техники, учёт поломок и расхода топлива

Исходные данные:

Необходимо реализовать веб-приложение для симуляции процесса снегоуборки в городской среде. Приложение должно импортировать граф дорожной сети из OpenStreetMap, строить маршруты уборки, запускать пошаговую симуляцию движения снегоуборочной техники с учётом расхода топлива, загрузки снега, поломок и ремонта, а также предоставлять аналитику и статистику по результатам симуляции.

Содержание пояснительной записки:

«Содержание»
«Введение»
«Качественные требования к решению»
«Сценарий использования»
«Модель данных»
«Разработка приложения»
«Вывод»
«Приложение»

Предполагаемый объем пояснительной записки:

Не менее 41 страницы.

Дата выдачи задания: 13.02.2026

Дата сдачи реферата: 20.05.2026

Дата защиты реферата: 20.05.2026

Студенты гр. 3343

Старков С. А

Поддубный В.А.

Жучков О. Д.

Преподаватель

Заславский М.М.

АННОТАЦИЯ

Проект «Snow Cleaning Simulation» представляет собой веб-приложение для симуляции процесса снегоуборки на улично-дорожной сети города Санкт-Петербурга. Система предоставляет возможность импортировать граф дорог из OpenStreetMap, создавать маршруты уборки с промежуточными точками, запускать пошаговую симуляцию движения снегоуборочной техники с учётом расхода топлива, загрузки снега, выгрузки на снегоплавильные пункты, заправки, поломок и ремонта, а также анализировать результаты симуляции с использованием статистических графиков.

В рамках индивидуального домашнего задания был проведён сравнительный анализ двух подходов к хранению данных: графового (Neo4j) и реляционного (PostgreSQL). Для обеих моделей выполнена оценка удельного объёма памяти, уровня избыточности данных и количества запросов, необходимых для реализации основных сценариев использования.

Результаты анализа показали, что для рассматриваемой предметной области более подходящей является графовая модель хранения данных. Она обеспечивает естественное представление дорожной сети в виде узлов и связей, сокращает количество запросов при поиске кратчайших путей и позволяет эффективно моделировать сложные отношения между сущностями (принадлежность маршрута симуляции, состояния техники на каждом шаге).

В проекте используются следующие технологии: Python, FastAPI, Neo4j, Redis, Leaflet, Chart.js, Docker и др.

Исходный код проекта и дополнительная информация доступны в репозитории: <https://github.com/moevm/nsql1h26-snow>

ANNOTATION

The «Snow Cleaning Simulation» project is a web application for simulating the snow removal process on the road network of Saint Petersburg. The system provides the ability to import a road graph from OpenStreetMap, create cleaning routes with waypoints, run step-by-step simulation of snow removal vehicles taking into account fuel consumption, snow loading, dumping at snow melting points, refueling, breakdowns and repairs, as well as analyze simulation results using statistical charts.

As part of an individual homework assignment, a comparative analysis of two approaches to data storage was conducted: graph (Neo4j) and relational (PostgreSQL). For both models, the specific amount of memory, the level of data redundancy, and the number of queries required to implement the main usage scenarios were estimated.

The results of the analysis showed that the graph data storage model is more suitable for the subject area under consideration. It provides a natural representation of the road network as nodes and edges, reduces the number of queries when finding shortest paths, and allows efficient modeling of complex relationships between entities (route belonging to a simulation, vehicle states at each step).

The application uses these instruments: Python, FastAPI, Neo4j, Redis, Leaflet, Chart.js, Docker etc.

The project's source code and additional information are available in the repository: <https://github.com/moevm/nsql1h26-snow>.

Оглавление

1. Введение	6
2. Сценарии использования	8
3. Модель данных	15
4. Разработанное приложение	29
5. Выводы	36
Литература	38
Приложения	39

1. Введение

Целью работы является создание веб-приложения «Snow Cleaning Simulation», предназначенного для симуляции процесса уборки снега на улично-дорожной сети города. Приложение ориентировано на специалистов дорожно-эксплуатационных служб, диспетчеров и аналитиков городского хозяйства. В ходе выполнения ИДЗ необходимо спроектировать модель данных, обосновать выбор СУБД и реализовать прототип с базовым набором функций.

а. Актуальность решаемой проблемы

Ежегодно в городах с суровым климатом, таких как Санкт-Петербург, уборка снега с улиц и дорог является критически важной задачей. Неэффективная логистика снегоуборочной техники приводит к образованию наледи, заторам и аварийным ситуациям. Существующие решения для управления уборкой дороги либо избыточно сложны и дороги, либо не позволяют гибко моделировать различные сценарии с учётом поломок техники, расхода топлива, объёма снега и инфраструктуры (парковки, снегоплавильные пункты, сервисные станции). Требуется лёгкое настраиваемое решение, в котором пользователь может самостоятельно загружать граф дорог, создавать маршруты уборки, запускать симуляцию и получать аналитику по результатам.

б. Постановка задачи

Разработать веб-приложение для симуляции процесса снегоуборки на дорожной сети. Приложение должно обеспечивать регистрацию и аутентификацию пользователей, а также предоставлять возможность импорта графа дорожной сети из данных OpenStreetMap (OSM). Пользователь может создавать маршруты уборки с указанием начальной, конечной и промежуточных точек. Система должна поддерживать запуск пошаговой симуляции, моделирующей движение снегоуборочной техники с учётом расхода топлива, загрузки и выгрузки снега, поломок и ремонта, а также маршрутизации между объектами инфраструктуры (парковки, снегоплавильные пункты, сервисные станции). Система должна обеспечивать отображение статистики и аналитики по результатам симуляции, а также импорт и экспорт данных. Для хранения данных предполагается использование Neo4j.

с. Предлагаемое решение

Серверная часть приложения реализована с использованием Python и FastAPI. Бизнес-логика симуляции вынесена в отдельный движок (SimulationEngine), реализующий пошаговую модель с конечными автоматами состояний техники. В качестве системы хранения данных используется графовая СУБД Neo4j, естественным образом моделирующая

дорожную сеть (узлы-перекрёстки, связи-дороги) и отношения между сущностями (маршрут содержит точки, симуляция содержит маршруты, шаг содержит состояния техники). Кэширование состояния симуляции осуществляется через Redis. Клиентская часть реализована на ванильном JavaScript с использованием библиотеки Leaflet для отображения интерактивной карты и Chart.js для построения статистических графиков. Развёртывание приложения выполняется с использованием Docker Compose.

d. Качественные требования к решению

Работа с картой должна быть простой и понятной: пользователь должен иметь возможность задать маршрут уборки за несколько кликов. Симуляция должна корректно обрабатывать десятки единиц техники на сети из сотен дорог. Система должна оставаться гибкой и позволять добавлять новые типы объектов инфраструктуры и состояний техники без изменения структуры базы данных. Безопасность обеспечивается JWT-аутентификацией и хранением паролей в зашифрованном виде (bcrypt). Архитектура приложения должна обеспечивать удобство расширения и сопровождения системы, в том числе возможность замены отдельных компонентов. Также предусматривается возможность масштабирования Neo4j, включая создание кластера при увеличении нагрузки.

2. Сценарии использования

а. Макет UI

Приложение представляет собой веб-интерфейс для симуляции уборки снега в Санкт-Петербурге. Базовый макет строится вокруг карты и боковых панелей, актуальная версия дополнена отдельными страницами для маршрутов, точек и симуляций.

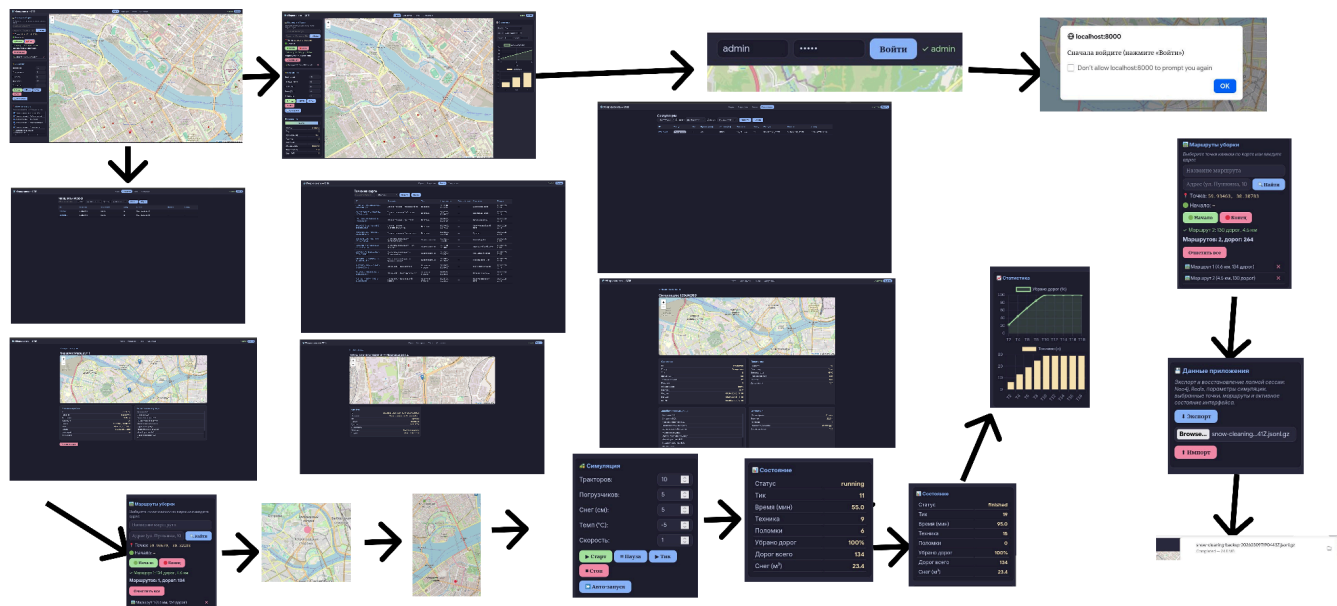


Рисунок 1 – страница входа/регистрации

Приложение представляет собой веб-интерфейс для симуляции уборки снега в Санкт-Петербурге. Базовый макет строится вокруг карты и боковых панелей, актуальная версия дополнена отдельными страницами для маршрутов, точек и симуляций.

Компоненты интерфейса

1. Верхняя навигация — переходы между картой, маршрутами, точками и симуляциями.
2. Центральная область — интерактивная карта Санкт-Петербурга (Leaflet), где строятся маршруты и показывается техника.
3. Левая панель (Sidebar) — создание маршрутов, управление симуляцией, просмотр статических объектов и текущего состояния.
4. Правая панель (Charts) — графики статистики, которые появляются во время симуляции.
5. Toast-уведомления — сообщения об успешных действиях и ошибках поверх карты.
6. Страницы списка и деталей — отдельный просмотр маршрутов, точек и симуляций без привязки к текущей сессии на карте

Левая панель содержит разделы.

- Маршруты уборки — ввод названия маршрута, выбор начальной и конечной точки на карте или через поиск адреса, кнопки «Начало» и «Конец».
- Симуляция — числовые параметры (тракторы, погрузчики, снег, температура, скорость), кнопки управления (Старт, Пауза, Тик, Стоп, Авто-запуск).

Дополнительные страницы приложения

- routes.html — список сохранённых маршрутов из Neo4j.
- route.html?id=... — один сохранённый маршрут с полным путём, улицами и метаданными.
- points.html — список точек инфраструктуры: парковки, снегоплавильни, сервисные станции.
- point.html?id=... — одна точка с мини-картой и подробной карточкой.
- simulations.html — история запусков симуляций.
- simulation.html?id=... — одна симуляция с сохранённым маршрутом, параметрами и статистикой.

Правая панель (при активной симуляции)

- Статистика — два графика: прогресс уборки (%) и расход топлива (л).
- Фильтры статистики — переключение осей и диапазона тиков для анализа динамики.

б. Сценарии использования для задачи

а. Импорта данных.

- Импорт данных (загрузка из JSON-дампа Neo4j)

Предусловие: пользователь авторизован и находится на главном экране приложения.

Основной сценарий:

1. Пользователь нажимает кнопку «Импорт/Экспорт» в навигационной панели.
2. В открывшемся диалоге нажимает кнопку «Импорт» и выбирает файл формата .json (APOC JSON dump).
3. Система проверяет формат файла и инициирует APOC-импорт в Neo4j.
4. При успешном импорте система обновляет граф дорожной сети, маршруты и объекты инфраструктуры.

Альтернативный сценарий:

- Файл имеет неподдерживаемый формат — пользователь видит сообщение об ошибке. Импорт не выполняется.
- Файл повреждён или содержит некорректные данные — система выводит уведомление «APOC import error».
- Пользователь закрывает диалог выбора файла — импорт прекращается, никаких изменений в базе данных не происходит.

Результат: данные успешно добавлены в систему и доступны пользователю для просмотра, создания маршрутов и запуска симуляций.

Импорт данных (загрузка из OpenStreetMap)

Предусловие: в файле .env установлено IMPORT_JSON=FALSE, приложение запущено через Docker Compose.

Основной сценарий:

- При запуске приложение автоматически скачивает PBF-файл с данными СЗФО из Geofabrik.
- С помощью osmium-tool вырезается область Аптекарского острова по заданному bounding box (30.2970, 59.9665, 30.3305, 59.9805).
- Система парсит OSM XML, извлекает проезжие дороги и перекрёстки, импортирует их в Neo4j в виде узлов Point и связей ROAD.

Результат: граф дорожной сети Аптекарского острова загружен в базу данных.

Ручной ввод данных (создание маршрутов и объектов инфраструктуры)

Предусловие: пользователь авторизован и находится на главном экране приложения.

Основной сценарий (создание маршрута уборки):

1. Пользователь нажимает на карту для выбора точки, нажимает кнопку «Начало» для задания начальной точки маршрута.
2. Добавляет промежуточные точки кнопкой «+Точка», нажимает «Конец» для задания конечной точки.
3. Вводит название маршрута в поле «Название маршрута».
4. Нажимает «Добавить маршрут». Система прокладывает кратчайший путь по графу дорог и сохраняет маршрут.

Ручное добавление объектов инфраструктуры:

1. Пользователь переходит на страницу «Объекты», нажимает «Добавить объект».
2. Выбирает тип: парковка, снегоплавильный пункт или сервисная станция.
3. Вводит название, координаты (кликом на карте или вручную), ёмкость (опционально), описание.
4. Нажимает «Создать».

Альтернативный сценарий:

- Не заполнены обязательные поля — система выводит предупреждение.
- Точки маршрута не соединены дорогами в графе — система выводит сообщение о невозможности построить путь.
- Пользователь нажимает «Отмена» — все введённые данные теряются, объект не создаётся.

Результат: новый маршрут или объект инфраструктуры успешно создан и визуализирован на карте.

i. Представления данных.

Отображение дорожной сети на карте: после загрузки графа дорог все перекрёстки и дороги автоматически подгружаются через API. Дороги отображаются линиями на карте Leaflet с цветовой индикацией статуса (очищена/не очищена). Маршруты уборки рисуются в виде выделенных линий. Объекты инфраструктуры отображаются маркерами на карте.

Табличное представление: на страницах «Маршруты», «Симуляции», «Объекты», «Шаги», «Состояния техники», «Пользователи» данные представлены в виде таблиц с серверной пагинацией, фильтрацией и настраиваемой статистикой (максимум, минимум, среднее, сумма по столбцам).

Поиск данных: пользователь вводит поисковые запросы и фильтры на каждой странице (по названию, типу, дате, расстоянию и др.). Система выполняет поиск и возвращает список совпадений с пагинацией.

Альтернативные варианты: если поисковый запрос не дал результатов — отображается пустая таблица с сообщением «Нет данных».

ii. Анализа данных.

Предусловие: пользователь авторизован и имеется хотя бы одна завершённая или запущенная симуляция.

Основной сценарий:

1. Пользователь переходит на страницу симуляции или в раздел статистики.
2. Система отображает сводные метрики: процент очищенных дорог, собранный снег (м^3), израсходованное топливо (л), количество поломок, стоимость ремонта, эффективность.
3. Пользователь может просмотреть графики изменения метрик по шагам симуляции (Chart.js).

Альтернативный сценарий:

- Симуляция ещё не запущена — вместо статистики отображается начальное состояние.
- Нет данных для выбранной комбинации фильтров — отображается сообщение об отсутствии данных.

Результат: на экране отображается интерактивная статистика, позволяющая оценить эффективность уборки снега

i. Экспорта данных.

Предусловие: пользователь авторизован.

Основной сценарий:

1. Пользователь нажимает кнопку «Импорт/Экспорт» в навигационной панели.
2. Нажимает кнопку «Экспорт».

3. Сервер извлекает все данные из Neo4j с помощью APOC, формирует JSON-файл и отправляет его клиенту.
4. Браузер начинает загрузку файла на устройство пользователя.

Альтернативный сценарий:

1. Ошибка соединения — появляется сообщение об ошибке экспорта.

Результат: файл с полной копией базы данных сохранён на локальном устройстве в формате APOC JSON, готовый для импорта в другой экземпляр Neo4j.

с. Вывод о преобладающих операциях

В системе доминируют операции чтения: просмотр дорожного графа, отображение состояний техники, получение статистики симуляции. Запись происходит при создании маршрутов, запуске симуляции и фиксации каждого шага. Однако поиск кратчайших путей — ключевая операция, требующая обхода графа, что делает графовую модель особенно выгодной. Это оправдывает выбор Neo4j, оптимизированного для графовых обходов и навигационных запросов.

3. Модель данных

а. Нереляционная модель данных Neo4j

і. Графическое представление

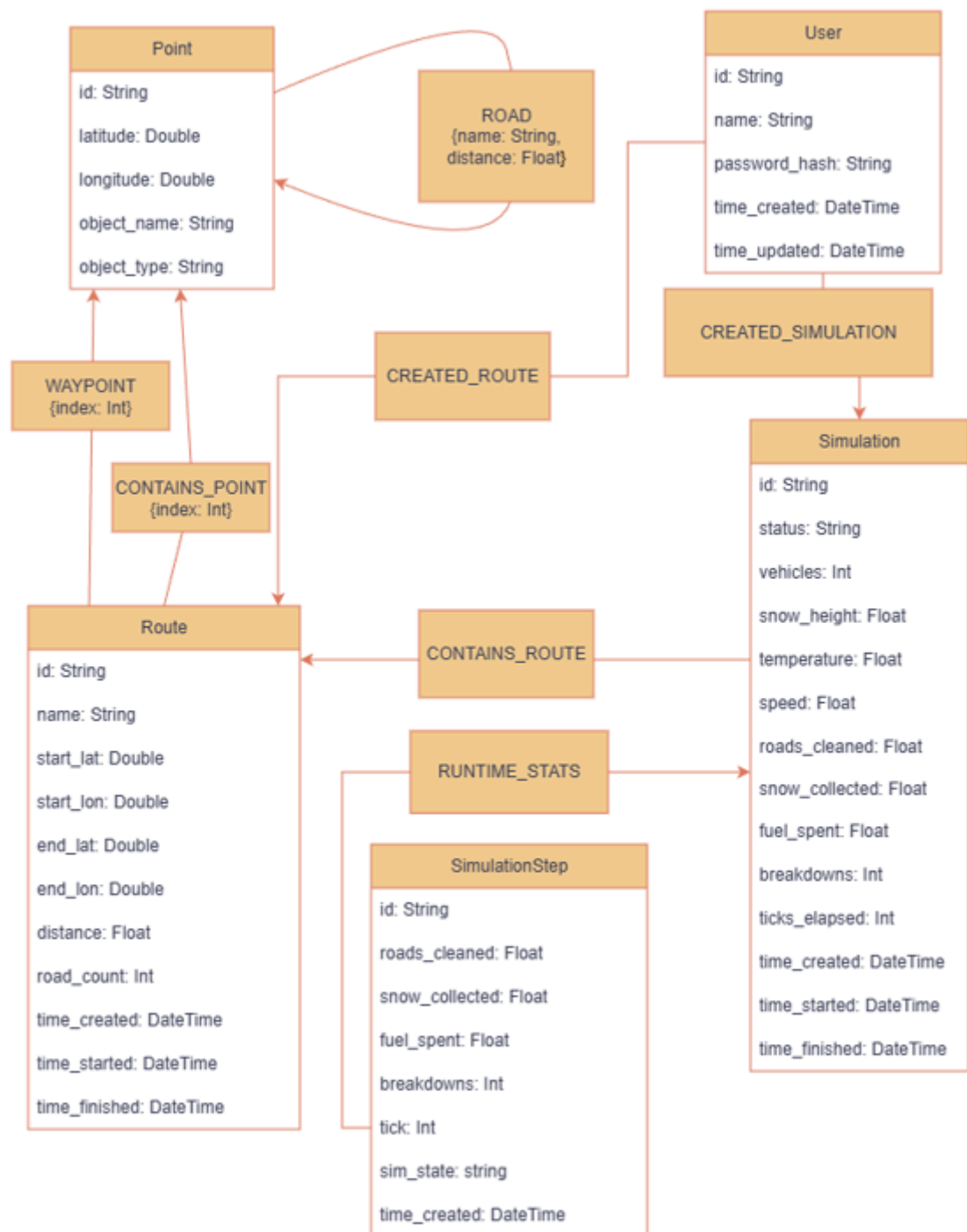


Рисунок 11 – нереляционная модель БД

іі. Описание назначений коллекций, типов данных и сущностей

Узлы:

Point — перекрёстки и объекты инфраструктуры. Узел хранит данные точек дорожной сети. Каждый узел соответствует одному перекрёстку или объекту инфраструктуры. Содержит свойства:

- `id` — уникальный идентификатор точки, строка
- `lat` — широта, число с плавающей точкой
- `lng` — долгота, число с плавающей точкой
- `object_type` — тип объекта инфраструктуры (`parking`, `snow_polygon`, `service_station`), `NULL` для обычных перекрёстков
- `object_name` — название объекта инфраструктуры, строка (опционально)
- `capacity` — ёмкость объекта, целое число (опционально)
- `description` — описание объекта, строка (опционально)
- `created_at` — дата создания
- `updated_at` — дата редактирования

Route — маршруты уборки. Узел хранит данные маршрутов, созданных пользователем. Содержит свойства:

- `id` — уникальный идентификатор маршрута, строка
- `label` — название маршрута, строка
- `start_lat`, `start_lng` — координаты начала маршрута
- `end_lat`, `end_lng` — координаты конца маршрута
- `path_nodes_json` — JSON-массив координат пути маршрута
- `streets` — список улиц маршрута, массив строк
- `distance_m` — общая длина маршрута в метрах
- `created_at` — дата создания
- `updated_at` — дата редактирования
- `started_at` — дата начала уборки по маршруту
- `finished_at` — дата завершения уборки по маршруту

Simulation — симуляции. Узел хранит данные запусков симуляции. Содержит свойства:

- `id` — уникальный идентификатор симуляции, строка
- `name` — название симуляции, строка (опционально)
- `status` — статус симуляции (`idle`, `running`, `paused`, `finished`, `error`)
- `tick` — текущий шаг симуляции, целое число
- `elapsed_minutes` — прошедшее модельное время в минутах
- `vehicles_total` — общее количество техники
- `vehicles_active` — количество активной техники
- `vehicles_broken` — количество сломанной техники

- `vehicles_en_route` — количество техники в пути
- `vehicles_cleaning` — количество убирающей техники
- `vehicles_dumping` — количество выгружающей снег техники
- `vehicles_refueling` — количество заправляющейся техники
- `vehicles_maintenance` — количество техники на обслуживании
- `roads_cleaned_pct` — процент очищенных дорог
- `snow_collected_m3` — собранный снег в куб. метрах
- `fuel_spent_l` — израсходованное топливо в литрах
- `avg_fuel_pct` — средний уровень топлива в процентах
- `avg_snow_load_pct` — средняя загрузка снегом в процентах
- `streets` — список улиц, массив строк
- `route_coords_json` — JSON-массив координат маршрутов
- `params_json` — JSON-объект параметров симуляции
- `roads_total` — общее количество дорог
- `created_at` — дата создания
- `updated_at` — дата редактирования
- `started_at` — дата запуска
- `finished_at` — дата завершения

`SimulationStep` — шаги симуляции. Узел хранит снимок состояния симуляции на конкретном шаге. Содержит свойства:

- `id` — уникальный идентификатор шага, строка
- `tick` — номер шага
- `roads_cleaned` — количество очищенных дорог на шаге
- `snow_collected` — собранный снег на шаге
- `fuel_spent` — израсходованное топливо на шаге
- `breakdowns` — количество поломок на шаге
- `sim_state` — JSON-снимок состояния симуляции
- `created_at` — дата создания
- `updated_at` — дата редактирования

`VehicleState` — состояния техники. Узел хранит данные о состоянии единицы техники на конкретном шаге. Содержит свойства:

- `id` — уникальный идентификатор состояния, строка

- machine_id — идентификатор машины
- vehicle_type — тип техники (tractor)
- status — текущий статус (idle, en_route, off_route, cleaning, dumping, refueling, broken, maintenance)
- lat, lng — текущие координаты
- home_parking_id, home_parking_lat, home_parking_lng — парковка приписки
- fuel_level — текущий уровень топлива, литры
- snow_loaded_m3 — загруженный снег, куб. метры
- distance_travelled_km — пройденное расстояние, км
- speed_kmh — текущая скорость
- travel_speed_kmh — скорость перемещения, км/ч
- cleaning_speed_kmh — скорость уборки, км/ч
- fuel_consumption_l_per_km — расход топлива, л/км
- fuel_capacity_l — ёмкость бака, литры
- snow_capacity_m3 — ёмкость кузова, куб. метры
- breakdown_probability — вероятность поломки за шаг
- repair_time_min — время ремонта, минуты
- repair_remaining_min — оставшееся время ремонта
- target_type — тип текущей цели (road_start, snow_polygon, service_station_refuel, service_station_maintenance, parking, road_resume)
- target_id — идентификатор целевого объекта
- target_lat, target_lng — координаты цели
- road_target_lat, road_target_lng — координаты конца убираемого участка
- road_resume_lat, road_resume_lng — координаты точки возобновления уборки
- progress_m — прогресс уборки текущего участка, метры
- current_edge — текущее ребро дороги (format: "src_id->dst_id")
- current_road — название текущей улицы
- created_at — дата создания
- updated_at — дата редактирования

User — пользователи. Узел хранит данные пользователей системы. Содержит свойства:

- id — уникальный идентификатор пользователя, строка

- name — логин пользователя
- password_hash — хеш пароля (bcrypt)
- role — роль пользователя (admin, operator)
- created_at — дата создания
- updated_at — дата редактирования

AppMeta — метаданные приложения. Узел хранит служебную информацию. Содержит свойства:

- key — ключ метаданных, строка
- value — значение, строка
- updated_at — дата обновления

Связи:

ROAD — дорога между перекрёстками. Связь от Point к Point. Содержит свойства:

2. distance — длина дороги в метрах
3. name — название улицы
4. highway — тип дороги (residential, primary, secondary и др.)
5. oneway — признак одностороннего движения, логический
6. geometry — JSON-массив координат геометрии дороги
7. cleaned — признак очищенности, логический
8. snow_cm — высота снега в сантиметрах
9. cleaned_m — протяжённость очищенного участка в метрах

CONTAINS_POINT — принадлежность точки маршруту. Связь от Route к Point. Содержит свойства:

- index — порядковый номер точки в маршруте

WAYPOINT — промежуточная точка маршрута. Связь от Route к Point. Содержит свойства:

- role — роль точки (start, waypoint, end)
- index — порядковый номер точки

CONTAINS_ROUTE — принадлежность маршрута симуляции. Связь от Simulation к Route.

RUNTIME_STATS — принадлежность шага симуляции. Связь от SimulationStep к Simulation. Содержит свойства:

2. index — порядковый номер шага

VEHICLE_DETAILS — принадлежность состояния техники шагу. Связь от VehicleState к SimulationStep. Содержит свойства:

1. index — порядковый номер

CREATED_ROUTE — создание маршрута пользователем. Связь от User к Route.

CREATED_SIMULATION — создание симуляции пользователем. Связь от User к Simulation.

2. Оценка удельного объема информации, хранимой в модели

Общий объем памяти, необходимый для хранения всех данных, зависит от количества дорог N (связи ROAD), так как дорожная сеть является центральной сущностью системы. Остальные сущности связаны с дорогами через средние статистические коэффициенты, полученные на основе типичного использования:

1. Количество объектов инфраструктуры (Point с object_type): $N/50$ (1 объект на 50 дорог)
2. Количество маршрутов: $N/20$ (1 маршрут на 20 дорог)
3. Количество симуляций: $N/100$ (1 симуляция на 100 дорог)
4. Количество шагов симуляции: $50S$ (50 шагов на симуляцию, где $S = N/100$)
5. Количество состояний техники: $80 * 50S$ (80 машин на 50 шагов)
6. Количество пользователей: $N/200$
7. Среднее количество точек маршрута: $10R$ (10 точек на маршрут)

Накладные расходы, учтенные при расчёте объема БД (в байтах):

1. Neo4j node record: 15 байт фиксированная часть + динамические свойства. Свойства хранятся в отдельных записях property store, каждая запись ~32 байта для простых типов, строки — по 1 байту на символ + заголовок.
2. Neo4j relationship record: 34 байт фиксированная часть + динамические свойства.
3. Связь ROAD с 7 свойствами: $\sim 34 + 7 * 32$ + длина строк (name ~30 байт, geometry ~200 байт) ≈ 498 байт.
4. Узел Point (перекресток) с 4 свойствами: $\sim 15 + 4 * 32 \approx 143$ байта.
5. Узел Point (инфраструктура) с 8 свойствами: $\sim 15 + 8 * 32$ + строковые поля ~100 байт ≈ 371 байт.
6. Узел Route с 10 свойствами: $\sim 15 + 10 * 32$ + path_nodes_json ~500 байт + streets ~100 байт ≈ 935 байт.
7. Узел Simulation с 20+ свойствами: $\sim 15 + 20 * 32$ + route_coords_json ~2000 байт + params_json ~500 байт ≈ 3155 байт.
8. Узел SimulationStep с 7 свойствами: $\sim 15 + 7 * 32$ + sim_state ~3000 байт ≈ 3239 байт.
9. Узел VehicleState с 30+ свойствами: $\sim 15 + 30 * 32 \approx 975$ байт.
10. Узел User с 5 свойствами: $\sim 15 + 5 * 32$ + password_hash ~60 байт ≈ 235 байт.
11. Накладные расходы на индексы: B-tree индексы по id, lat/lng, object_type — добавляют ~10-15% к объёму.
12. Суммарно добавляем ~15% к «чистому» объёму.

Суммарный объём вычисляется как сумма произведений количества записей на их размер. После подстановки всех выражений и упрощения получаем линейную зависимость от N:

$$V(N) = 1.15 * (1.2N * 143 + N * 498 + N/50 * 371 + N/20 * 935 + N/100 * 3155 + 50 * N/100 * 3239 + 80 * 50 * N/100 * 975 + N/200 * 235) \text{ байт}$$

V(N) = 1.e — название улицы (VARCHAR, NULLABLE)

- highway — тип дороги (VARCHAR, NULLABLE)
- oneway — признак односторонности (BOOLEAN)
- geometry — координаты геометрии (JSONB, NULLABLE)
- cleaned — признак очищенности (BOOLEAN)
- snow_cm — высота снега (DOUBLE PRECISION)
- cleaned_m — протяжённость очищенного участка (DOUBLE PRECISION)

Первичный ключ: (src_id, dst_id)

Route — маршрут уборки, сущность хранит данные маршрутов. Содержит поля:

- id — уникальный идентификатор, первичный ключ (VARCHAR)
- user_id — ссылка на пользователя, внешний ключ (VARCHAR)
- label — название маршрута (VARCHAR)
- start_lat, start_lng — координаты начала (DOUBLE PRECISION)
- end_lat, end_lng — координаты конца (DOUBLE PRECISION)
- path_nodes_json — координаты пути (JSONB)
- streets — список улиц (JSONB)
- distance_m — длина маршрута (DOUBLE PRECISION)
- created_at — дата создания (TIMESTAMP)
- updated_at — дата редактирования (TIMESTAMP, NULLABLE)
- started_at — дата начала (TIMESTAMP, NULLABLE)
- finished_at — дата завершения (TIMESTAMP, NULLABLE)

RoutePoint — точка маршрута, сущность хранит данные о точках, входящих в маршрут. Содержит поля:

- route_id — идентификатор маршрута, внешний ключ (VARCHAR)
- point_id — идентификатор перекрёстка, внешний ключ (VARCHAR)
- index — порядковый номер (INTEGER)
- role — роль точки (VARCHAR: start, waypoint, end)

Первичный ключ: (route_id, point_id, index)

RouteWaypoint — промежуточная точка маршрута. Содержит поля:

- route_id — идентификатор маршрута, внешний ключ (VARCHAR)
- point_id — идентификатор перекрёстка, внешний ключ (VARCHAR)
- role — роль (VARCHAR)
- index — порядковый номер (INTEGER)

Первичный ключ: (route_id, point_id)

Simulation — симуляция, сущность хранит данные запусков симуляции. Содержит поля:

- id — уникальный идентификатор, первичный ключ (VARCHAR)
- user_id — ссылка на пользователя, внешний ключ (VARCHAR)
- name — название (VARCHAR, NULLABLE)
- status — статус (VARCHAR: idle, running, paused, finished, error)
- tick — текущий шаг (INTEGER)
- elapsed_minutes — прошедшее время (DOUBLE PRECISION)
- vehicles_total, vehicles_active, vehicles_broken — количество техники (INTEGER)
- vehicles_en_route, vehicles_cleaning, vehicles_dumping, vehicles_refueling, vehicles_maintenance — количество техники по статусам (INTEGER)
- roads_cleaned_pct — процент очищенных дорог (DOUBLE PRECISION)
- snow_collected_m3 — собранный снег (DOUBLE PRECISION)
- fuel_spent_l — израсходованное топливо (DOUBLE PRECISION)
- avg_fuel_pct, avg_snow_load_pct — средние показатели (DOUBLE PRECISION)
- streets — список улиц (JSONB)
- route_coords_json — координаты маршрутов (JSONB)
- params_json — параметры симуляции (JSONB)
- roads_total — общее количество дорог (INTEGER)
- created_at, updated_at, started_at, finished_at — даты (TIMESTAMP)

SimulationRoute — связь симуляции с маршрутом. Содержит поля:

- simulation_id — внешний ключ (VARCHAR)
- route_id — внешний ключ (VARCHAR)

Первичный ключ: (simulation_id, route_id)

SimulationStep — шаг симуляции. Содержит поля:

- id — уникальный идентификатор, первичный ключ (VARCHAR)

- simulation_id — внешний ключ (VARCHAR)
- tick — номер шага (INTEGER)
- roads_cleaned, breakdowns — показатели шага (INTEGER)
- snow_collected, fuel_spent — показатели шага (DOUBLE PRECISION)
- sim_state — снимок состояния (JSONB)
- created_at, updated_at — даты (TIMESTAMP)

VehicleState — состояние техники. Содержит поля:

- id — уникальный идентификатор, первичный ключ (VARCHAR)
- step_id — внешний ключ на SimulationStep (VARCHAR)
- machine_id — идентификатор машины (VARCHAR)
- vehicle_type — тип техники (VARCHAR)
- status — статус (VARCHAR)
- lat, lng — координаты (DOUBLE PRECISION)
- home_parking_id — парковка приписки (VARCHAR, NULLABLE)
- home_parking_lat, home_parking_lng — координаты парковки (DOUBLE PRECISION, NULLABLE)
- fuel_level, snow_loaded_m3, distance_travelled_km, speed_kmh — числовые показатели (DOUBLE PRECISION)
- travel_speed_kmh, cleaning_speed_kmh — скорости (DOUBLE PRECISION)
- fuel_consumption_l_per_km, fuel_capacity_l, snow_capacity_m3 — параметры (DOUBLE PRECISION)
- breakdown_probability, repair_time_min, repair_remaining_min — вероятности и время (DOUBLE PRECISION)
- target_type, target_id — текущая цель (VARCHAR, NULLABLE)
- target_lat, target_lng — координаты цели (DOUBLE PRECISION, NULLABLE)
- road_target_lat, road_target_lng — координаты конца уборки (DOUBLE PRECISION, NULLABLE)
- road_resume_lat, road_resume_lng — точка возобновления (DOUBLE PRECISION, NULLABLE)
- progress_m — прогресс уборки (DOUBLE PRECISION)
- current_edge, current_road — текущий участок (VARCHAR, NULLABLE)
- created_at, updated_at — даты (TIMESTAMP)

User — пользователь. Содержит поля:

- id — уникальный идентификатор, первичный ключ (VARCHAR)
- name — логин (VARCHAR)
- password_hash — хеш пароля (VARCHAR)
- role — роль (VARCHAR)
- created_at, updated_at — даты (TIMESTAMP)

i. Оценка удельного объема информации, хранимой в модели

Общий принцип оценки: в реляционной модели данные хранятся в таблицах фиксированной структуры. В отличие от графовой модели, здесь связи между сущностями моделируются через внешние ключи и дополнительные таблицы связей. Размер строковых полей оценивается исходя из максимальной длины и кодировки UTF-8. Числовые поля имеют фиксированный размер.

Общий объём памяти зависит от количества дорог N :

- Количество перекрёстков: $\sim 1.2N$
- Количество объектов инфраструктуры: $N/50$
- Количество маршрутов: $N/20$
- Количество точек маршрута: $10 * N/20 = N/2$
- Количество точек маршрута (waypoints): $N/4$
- Количество симуляций: $N/100$
- Количество связей маршрут-симуляция: $N/20$
- Количество шагов: $50 * N/100 = N/2$
- Количество состояний техники: $80 * N/2 = 40N$
- Количество пользователей: $N/200$

Размеры записей (в байтах):

- Intersection — ~ 56
- MapObject — ~ 300
- Road — ~ 560
- Route — ~ 1200
- RoutePoint — ~ 70
- RouteWaypoint — ~ 70
- Simulation — ~ 1800
- SimulationRoute — ~ 30
- SimulationStep — ~ 3200
- VehicleState — ~ 900

- User — ~250

Учёт накладных расходов: индексы (PK, FK) → ~10-15%, выравнивание данных → ~5%, служебные структуры → ~5%. Итого добавляем ~20% к объёму.

Суммарный объём:

$$V(N) = 1.2 * (1.2N * 56 + N/50 * 300 + N * 560 + N/20 * 1200 + N/2 * 70 + N/4 * 70 + N/100 * 1800 + N/20 * 30 + N/2 * 3200 + 40N * 900 + N/200 * 250)$$

$$V(N) = 1.2 * (67.2N + 6N + 560N + 60N + 35N + 17.5N + 18N + 1.5N + 1600N + 36000N + 1.25N)$$

$$V(N) = 1.2 * 38366N \text{ байт} \approx 0.046N \text{ Мбайт}$$

Запросы к модели, с помощью которых реализуются сценарии использования

- Сценарий: поиск кратчайшего пути между двумя перекрёстками

В PostgreSQL отсутствует встроенный алгоритм кратчайшего пути. Необходима рекурсивная реализация (CTE WITH RECURSIVE) или расширение pgRouting.

С помощью pgRouting: *SELECT FROM pgr_dijkstra('SELECT src_id AS source, dst_id AS target, distance AS cost FROM "Road"', 'start_id', 'end_id');**

Для выполнения сценария требуется 1 запрос (с расширением pgRouting). Без расширения — сложная рекурсивная CTE, производительность значительно ниже. Количество задействованных таблиц: 1 (Road).

- Сценарий: поиск ближайшего перекрёстка к координатам

SELECT id, lat, lng, earth_distance(ll_to_earth(lat, lng), ll_to_earth(\$lat, \$lng)) AS dist FROM "Intersection" ORDER BY dist LIMIT 1;

Для выполнения сценария требуется 1 запрос. Количество задействованных таблиц: 1 (Intersection).

- Сценарий: создание маршрута уборки

INSERT INTO "Route" (id, user_id, label, start_lat, start_lng, end_lat, end_lng, path_nodes_json, streets, distance_m, created_at, updated_at) VALUES (\$id, \$user_id, \$label, \$start_lat, \$start_lng, \$end_lat, \$end_lng, \$path_nodes_json, \$streets, \$distance_m, NOW(), NOW());

И добавление точек маршрута: *INSERT INTO "RoutePoint" (route_id, point_id, index, role) VALUES (\$route_id, point_id_1, 0, 'start'), (\$route_id, \$point_id_2, 1, 'waypoint'), ...;*

Для выполнения сценария требуется 2 запроса. Количество задействованных таблиц: 2 (Route, RoutePoint).

- Сценарий: запуск симуляции и создание шага

INSERT в Simulation — 1 запрос. INSERT в SimulationRoute (связи с маршрутами) — 1 запрос (batch). INSERT в SimulationStep — 1 запрос. INSERT в VehicleState (80 машин) — 1 запрос (batch).

Для выполнения сценария требуется ~4 запроса. Количество задействованных таблиц: 4 (Simulation, SimulationRoute, SimulationStep, VehicleState).

- Сценарий: обновление прогресса уборки дороги

UPDATE "Road" SET cleaned_m = \$cleaned_m, cleaned = CASE WHEN \$cleaned_m + 0.1 >= distance THEN true ELSE cleaned END, snow_cm = CASE WHEN \$cleaned_m + 0.1 >= distance THEN 0.0 ELSE snow_cm END WHERE src_id = \$src AND dst_id = \$dst;

Для выполнения сценария требуется 1 запрос. Количество задействованных таблиц: 1 (Road).

- Сценарий: получение статистики симуляции

SELECT id, roads_cleaned_pct, snow_collected_m3, fuel_spent_l, vehicles_broken, elapsed_minutes FROM "Simulation" WHERE id = \$sim_id;

Для выполнения сценария требуется 1 запрос. Количество задействованных таблиц: 1 (Simulation).

а. Сравнение моделей

і. Удельный объем

Объём данных в обеих моделях зависит от количества дорог N , поскольку дорожная сеть является центральной сущностью системы.

- Для графовой модели получена формула общего объёма данных: $V(N) \approx 0.0476 \cdot N$ Мбайт
- Для реляционной модели: $V(N) \approx 0.046 \cdot N$ Мбайт

Объём хранения в моделях практически одинаковый. Реляционная модель занимает немного меньше памяти, однако разница незначительна. Причины различий:

- В графовой модели: узлы и связи хранят свойства в динамических хранилищах, JSON-поля (path_nodes_json, sim_state, params_json) занимают значительное место.
- В реляционной модели: данные распределены по таблицам, JSON-поля также используются (JSONB), но отсутствуют накладные расходы на хранение связей как отдельных записей.

Обе модели демонстрируют линейный рост объёма данных, а различия по памяти находятся в пределах погрешности оценки.

і. Запросы по отдельным юзкейсам

Важным критерием эффективности модели является количество запросов, необходимых для выполнения типичных операций системы.

Сценарии использования:

Поиск кратчайшего пути

- Графовая модель — 1 запрос (встроенный shortestPath)
- Реляционная модель — 1 запрос (pgRouting) или сложная рекурсивная CTE без расширения
- Neo4j имеет встроенную оптимизацию обхода графа, PostgreSQL требует установки pgRouting → графовая модель значительно эффективнее для навигационных запросов

Поиск ближайшего перекрёстка

- Графовая модель — 1 запрос (point.distance)
- Реляционная модель — 1 запрос (earth_distance или PostGIS)
- Модели эквивалентны по количеству запросов

Создание маршрута уборки

- Графовая модель — 2 запроса (CREATE Route + UNWIND CONTAINS_POINT)

- Реляционная модель — 2 запроса (INSERT Route + INSERT RoutePoint)
- Модели эквивалентны

Запуск симуляции

- Графовая модель — ~5-6 запросов
- Реляционная модель — ~4 запроса
- Реляционная модель компактнее за счёт batch INSERT, однако разница невелика

Обновление прогресса уборки

- Графовая модель — 1 запрос
- Реляционная модель — 1 запрос
- Модели эквивалентны

Получение статистики симуляции

- Графовая модель — 1 запрос
- Реляционная модель — 1 запрос

Модели эквивалентны

i. Вывод

Для данного приложения — системы симуляции уборки снега с графом дорожной сети, маршрутизацией техники и пошаговым моделированием — графовая модель (Neo4j) является более предпочтительной.

Обоснование:

- Дорожная сеть естественно моделируется как граф (узлы-перекрёстки, связи-дороги), что идеально соответствует парадигме Neo4j
- Поиск кратчайшего пути — ключевая операция, выполняемая на каждом шаге симуляции для каждой единицы техники; Neo4j оптимизирован для графовых обходов (shortestPath, GDS)
- Связи между сущностями (принадлежность маршрута симуляции, состояние техники на шаге) естественно выражаются через графовые отношения, без необходимости дополнительных таблиц связей
- Количество запросов при работе с графовыми структурами существенно меньше или сопоставимо
- Объём данных сопоставим с реляционной моделью
- Модель проще масштабируется при добавлении новых типов связей (например, «соседние дороги», «альтернативные маршруты»)

4. Разработанное приложение

I. Краткое описание

Разработанное приложение представляет собой распределённую клиент-серверную систему, состоящую из нескольких независимых сервисов, взаимодействующих между собой посредством HTTP-запросов. Все компоненты системы развёрнуты в Docker-контейнерах и объединены в единую инфраструктуру с помощью Docker Compose. Использование контейнеризации обеспечивает изоляцию сервисов, упрощает процесс развёртывания и повышает масштабируемость системы.

Сервисы и их роли:

- App (FastAPI) — серверная часть приложения, реализующая REST API и бизнес-логику. Выполняет обработку запросов, аутентификацию пользователей, валидацию данных, работу с базой данных Neo4j и движок симуляции.
- Neo4j — графовая база данных приложения. Хранит информацию о дорожной сети, маршрутах уборки, симуляциях, шагах, состояниях техники, объектах инфраструктуры и пользователях.
- Redis — кэш состояния симуляций. Обеспечивает быстрый доступ к текущему состоянию запущенных симуляций.
- Jaeger — система распределённой трассировки. Обеспечивает мониторинг производительности и отладку запросов через OpenTelemetry.

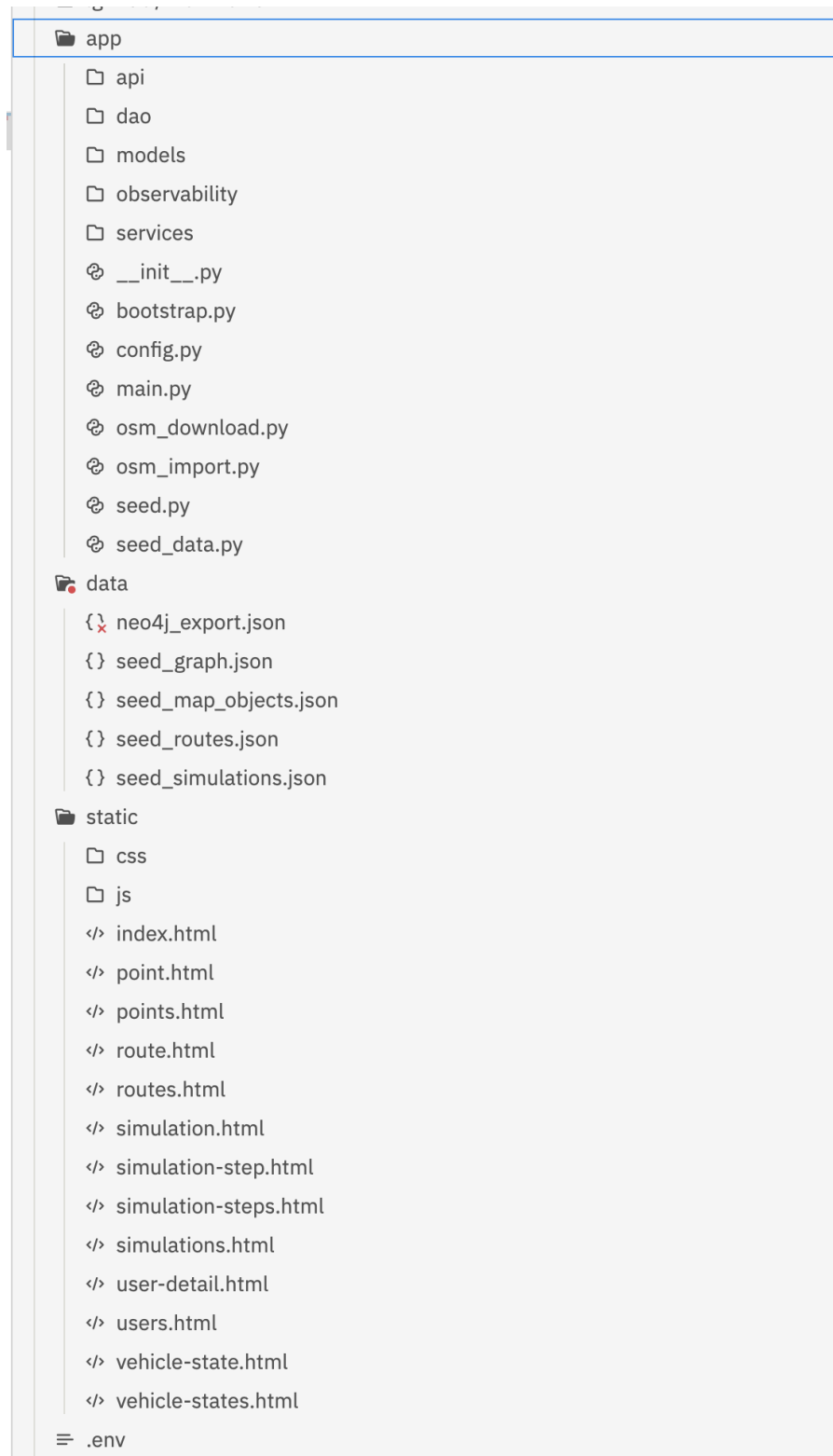


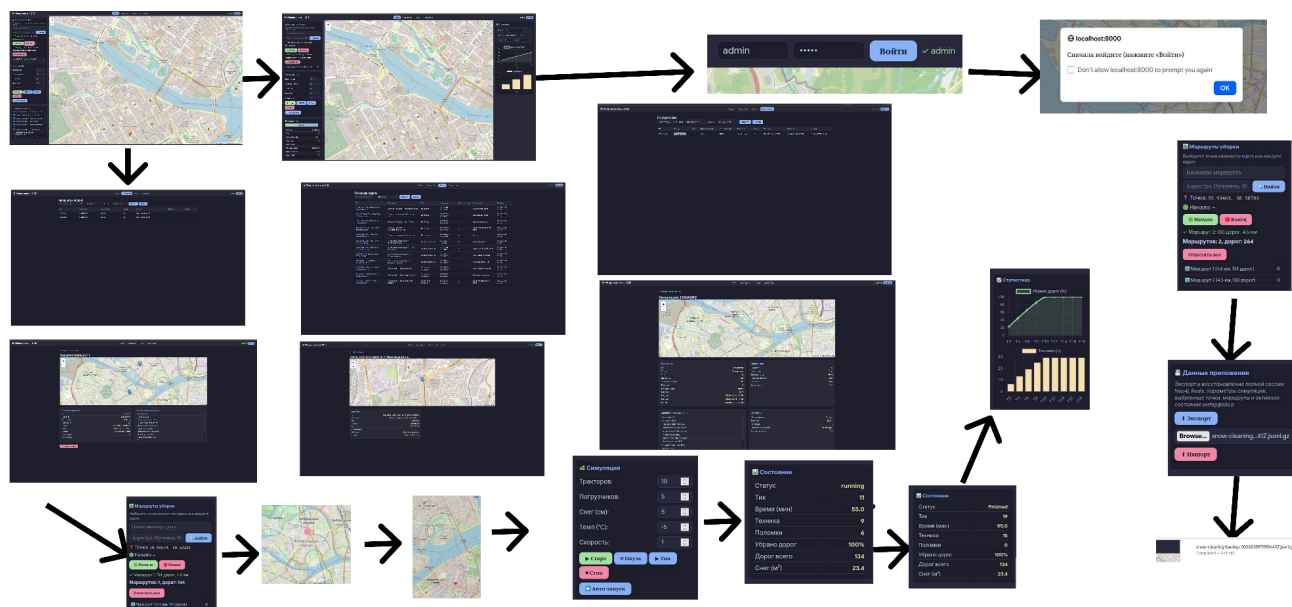
Рисунок 13 – структура проекта

а. Используемые технологии

Таблица 1. Используемые технологии

Слой	Технология	Обоснование выбора
Сервер	FastAPI (Python 3.12)	Высокопроизводительный async-фреймворк, автоматическая OpenAPI-документация, Pydantic-валидация из коробки
База данных	Neo4j 5.26 (Community)	Графовая модель естественно отражает структуру дорожной сети, встроенный shortestPath, GDS-плагин для графовых алгоритмов
Кэширование	Redis 7	Быстрое in-memory хранилище для кэширования состояния симуляций, снижение нагрузки на Neo4j
Аутентификация	python-jose + passlib/bcrypt	JWT-токены для stateless-авторизации, bcrypt для безопасного хеширования паролей
Картография	Leaflet + vanilla JS	Бесплатная, лёгкая библиотека для интерактивных карт, поддержка GeoJSON
Статистика	Chart.js	Интерактивные графики и диаграммы для визуализации метрик симуляции
Трассировка	OpenTelemetry + Jaeger	Стандартный протокол трассировки, визуализация распределённых запросов
Контейнеризация	Docker, Docker Compose	Изолированная среда, простота развёртывания на любой платформе
Импорт OSM	osmium-tool	Быстрый инструмент для извлечения bbox из PBF-файлов OpenStreetMap

в. Снимки экрана приложения



5. Выводы

6. Достигнутые результаты

Разработано веб-приложение «Snow Cleaning Simulation», предназначенное для симуляции процесса уборки снега на дорожной сети города. В приложении реализованы следующие основные возможности: импорт графа дорожной сети из OpenStreetMap или JSON-дампа; создание, редактирование и удаление маршрутов уборки с промежуточными точками; добавление и управление объектами инфраструктуры (парковки, снегоплавильные пункты, сервисные станции); пошаговая симуляция движения снегоуборочной техники с учётом расхода топлива, загрузки и выгрузки снега, поломок и ремонта; отображение статистики и аналитики с построением диаграмм; аутентификация пользователей и разграничение доступа к данным; REST API с автоматически формируемой документацией; импорт и экспорт данных в формате APOC JSON. Серверная часть приложения реализована на FastAPI с использованием Neo4j для хранения данных и Redis для кэширования, а клиентская часть разработана на ванильном JavaScript с Leaflet и Chart.js. Для контейнеризации и развёртывания приложения использовались Docker Compose и Jaeger.

Дополнительно был проведён сравнительный анализ Neo4j и PostgreSQL для хранения данных дорожной сети и симуляции. Результаты показали, что Neo4j является более подходящим решением для данной предметной области благодаря естественному представлению графа дорог, встроенным алгоритмам поиска кратчайших путей и эффективному моделированию отношений между сущностями.

б. Недостатки и пути для улучшения полученного решения

В системе отсутствует ролевая модель доступа: все аутентифицированные пользователи имеют одинаковый доступ ко всем данным. В дальнейшем возможно добавление разграничения прав доступа (администратор, диспетчер, наблюдатель).

Интерфейс приложения ориентирован на настольные устройства, мобильная версия требует адаптации элементов навигации и картографических элементов.

При работе с очень большим графом дорог (весь город) возможна деградация производительности поиска кратчайших путей. Для повышения эффективности могут использоваться GDS-алгоритмы Neo4j (Dijkstra с весами, A*) или предварительная кластеризация графа.

Текущая версия симуляции не поддерживает интеграцию с реальными данными о погоде и пробках (поля `use_real_weather` и `use_traffic` зарезервированы, но не реализованы).

Также текущая версия не поддерживает реальную загрузку и хранение изображений: поле `geometry` хранит JSON-строку с координатами, а не файлы.

с. Будущее развитие решения

В дальнейшем приложение может быть расширено за счёт интеграции с метеосервисами (OpenWeatherMap) для автоматической настройки высоты снегопада на основе реальных данных о погоде.

Перспективным направлением развития является интеграция с сервисами трафика (Яндекс.Пробки, TomTom) для учёта загруженности дорог при маршрутизации техники.

Для повышения удобства использования может быть реализована визуализация симуляции в реальном времени: автоматическое продвижение шагов с анимацией движения техники по карте.

Кроме того, планируется добавление поддержки нескольких типов снегоуборочной техники (тракторы, грузовики, погрузчики) с различными характеристиками скорости, ёмкости и расхода топлива.

Для масштабирования на весь город Санкт-Петербурга возможно использование кластера Neo4j (Causal Clustering) и шардирования данных по районам.

Литература

- a. Исходный код проекта: <https://github.com/moevm/nsql1h26-snow>
- b. Документация Neo4j: <https://neo4j.com/docs/>
- c. Neo4j Graph Data Science Library:
<https://neo4j.com/docs/graph-data-science/current/>
- d. FastAPI — modern, fast web framework for Python:
<https://fastapi.tiangolo.com/>
- e. Методические указания курса «NoSQL Введение»:
https://se.moevm.info/doku.php/staff:courses:no_sql_introduction:calculating_data_model_size
- f. Leaflet — JavaScript library for interactive maps: <https://leafletjs.com/>
- g. Docker Compose documentation: <https://docs.docker.com/compose/>
- h. OpenStreetMap Wiki: <https://wiki.openstreetmap.org/>
- i. osmium-tool: <https://osmcode.org/osmium-tool/>
- j. Chart.js documentation: <https://www.chartjs.org/docs/>

Приложения

А. Документация по сборке и разворачиванию приложения

Требования: Docker $\geq$ 20.10, Docker Compose $\geq$ 1.29.

Шаги для локального запуска:

- Клонировать репозиторий

```
git clone https://github.com/moevm/nsql1h26-snow.git
cd nsql1h26-snow
```

- Создайте конфигурационный файл из шаблона

```
cp .env.example .env
```

- Запустите сборку и все сервисы:

```
docker compose build --no-cache
docker compose up
```

При первом запуске будет выполнена загрузка OSM-данных и импорт тестовых данных. Возможна задержка в несколько минут.

- После успешного старта:

Приложение доступно по адресу <http://localhost:8000>

Neo4j Browser: <http://localhost:7474>

Jaeger UI: <http://localhost:16686>

Тестовые пользователи: admin:admin, operator:operator

- Альтернативный запуск с загрузкой из JSON-дампа (по умолчанию):

В .env установите IMPORT_JSON=TRUE (значение по умолчанию). При запуске будет импортирован предсобранный JSON-дамп из data/neo4j_export.json.

- Остановка приложения:

```
docker compose down
```

а. Инструкция для пользователя

Начало работы:

- Откройте приложение в браузере (<http://localhost:8000>). Войдите под тестовой учётной записью: admin:admin или operator:operator.

Создание маршрута уборки:

- На главной странице кликните по карте для выбора начальной точки маршрута.
- Нажмите кнопку «Начало» для фиксации начальной точки.
- Добавьте промежуточные точки кнопкой «+Точка» (необязательно).
- Нажмите «Конец» для фиксации конечной точки.
- Введите название маршрута в поле «Название маршрута».
- Нажмите «Добавить маршрут». Система проложит кратчайший путь по графу дорог.

Добавление объекта инфраструктуры:

- Перейдите на страницу «Объекты» через навигационную панель.

- Нажмите «Добавить объект».
- Выберите тип: парковка, снегоплавильный пункт или сервисная станция.
- Введите название, координаты (кликом на карте или вручную), ёмкость и описание.
- Нажмите «Создать».

Запуск симуляции:

- На главной странице в панели «Симуляция» добавьте маршруты уборки в список задач.
- Настройте параметры: количество техники (кнопка «Машины»), высоту снегопада (см), пороги топлива и загрузки.
- Введите название симуляции (необязательно).
- Нажмите «Запуск». Симуляция перейдёт в статус «running».
- Используйте кнопки «Шаг», «Пауза», «Возобновить», «Остановить» для управления симуляцией.

Просмотр статистики:

- Перейдите на страницу симуляции через список симуляций.
- Просмотрите сводные метрики: процент очищенных дорог, собранный снег, расход топлива, поломки, эффективность.
- Используйте графики Chart.js для анализа динамики по шагам.

Импорт/экспорт:

- Нажмите кнопку «Импорт/Экспорт» в навигационной панели.
- Для экспорта нажмите «Экспорт» — файл neo4j_export.json будет скачан.
- Для импорта нажмите «Импорт» — выберите APOC JSON файл для загрузки.

Управление пользователями:

- Перейдите на страницу «Пользователи» через навигационную панель.
- Просматривайте, создавайте, редактируйте и удаляйте учётные записи пользователей.